

Laporan Tugas Kecil 1 IF-2211 Strategi Algoritma
Semester II Tahun Akademik 2025/2026



Disusun Oleh:

Mahmudia Kimdaro Amin - 13524083

K-01

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI	1
DAFTAR GAMBAR	2
BAB I: PENDAHULUAN	3
1.1 Deskripsi Tugas	3
BAB II: PENYELESAIAN DAN ALGORITMA	4
2.1 Algoritma Brute Force	4
BAB III: STRUKTUR REPOSITORY DAN SOURCE CODE	5
3.1 Struktur Repository	5
3.2 Source Code	6
BAB IV: TESTING	8
4.1 Test case 1	8
4.2 Test case 2	8
4.3 Test case 3	8
4.4 Test case 4	9
4.5 Test case 5	10
LAMPIRAN	12
Github Repository	12
Miscellaneous (Tabel Poin)	12

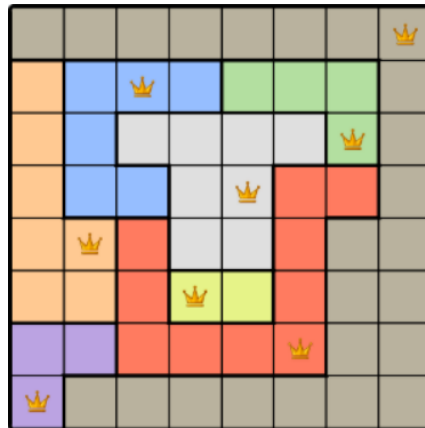
DAFTAR GAMBAR

<i>Gambar 1.</i> Papan Permainan IQ Puzzler Pro	4
<i>Gambar 2.</i> Kode Solver	

BAB I: PENDAHULUAN

1.1 Deskripsi Tugas

Queens Game adalah sebuah game logika yang tersedia pada website professional LinkedIn. Tujuan dari permainan ini adalah menaruh sebuah semua ratu pada sebuah papan persegi berwarna sehingga terdapat setiap ratu ada berada papan dan tidak melanggar aturan



Gambar 1. Papan Queens Game

Permainan dimulai dengan sebuah papan berukuran $N \times N$ dengan beberapa region warna sejumlah N . Untuk memenangi permainan Queens diperlukan untuk menaruh semua ratu pada sebanyak N pada papan. Aturan bagi ratu adalah,

1. Setiap ratu tidak boleh berada pada baris dan kolom yang sama
2. Setiap ratu tidak boleh berada pada region warna yang sama
3. Setiap ratu tidak boleh berada pada kotak yang bertetangaan dengan ratu lain termasuk secara diagonal

Tujuan dari tugas kecil ini adalah untuk menemukan cukup satu solusi dari permainan Queen game dengan menggunakan algoritma Brute-Force, atau menampilkan bahwa solusi tidak ditemukan jika tidak ada solusi yang mungkin dari puzzle tanpa menggunakan pendekatan heuristik.

BAB II: PENYELESAIAN DAN ALGORITMA

2.1 Algoritma Brute Force

Sebelum memulai melakukan Brute-force pertama harus melakukan parsing dari input yang merupakan txt file. File txt berisi text dengan N baris dan N buah karakter per baris sebagai representasi dari papan. Hasil dari text yang di parsing lalu dimasukkan ke dalam larik dua dimensi sebagai representasi dari papan. Larik lalu digunakan sebagai representasi dari papan dalam melakukan algoritma Brute-force. Berikut merupakan langkah dari cara kerja algoritma

1. Program membuat sebuah larik dua dimensi dengan 2 ruang untuk menyimpan koordinat x dan y sebanyak N
2. Program lalu mulai menaruh setiap ratu satu per satu pada salah satu kotak dari papan dan mencatat Lokasi ratu pada larik ratu.
3. Program akan melakukan iterasi dengan menaruh ratu satu per satu pada setiap kemungkinan kotak dimulai dari ratu pertama hingga ratu terakhir dengan melakukan cek apakah kotak tersebut sudah ditempati terlebih dahulu. Urutan penaruhan kotak dilakukan dengan mengisi dari (1,1,) (1,2) dan seterusnya hingga (N, N).
4. Setelah semua ratu telah diletakan pada papan, Program akan melakukan cek pada posisi ratu yang sekarang, program melihat apakah posisi ratu yang sekarang dapat digunakan sebagai Solusi.
5. Jika posisi terkini tidak bisa digunakan sebagai Solusi program lanjut mengecek kondisi selanjutnya dengan memindahkan ratu terakhir ke kotak selanjutnya.

Langkah tersebut adalah cara program melakukan Brute-force dengan mencoba semua kemungkinan konfigurasi dari ratu hingga ditemukan konfigurasi dari ratu yang merupakan Solusi dari papan. Bila tidak ditemukan solusi akan ditampilkan no solution found

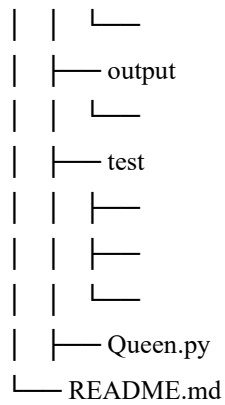
BAB III: STRUKTUR REPOSITORY DAN SOURCE CODE

3.1 Struktur Repository

Berikut adalah struktur *repository* dari program ini. Jika ingin menjalankan program dengan Command Line Interface (CLI), *input file* harus berada di
.../Tucil1_13524083/src/test/ dan berupa .txt

Tucil1_13523090/

```
├── src
│   ├── docs
│   ├── 
│   └── 
```



Berikut adalah penjelasan dari tiap folder:

1. Folder **src** berisi *source code* program.
2. Folder **test** berisi input uji coba
3. Folder output berisi solusi dari ini

3.2 Source Code

Berikut adalah *source code* dari program untuk solver ini yang berupa tangkapan layar, untuk lebih lengkapnya, *source code* dapat diakses pada link [berikut](#).

```

def plsSolveQueen(num, matrix, Queen):
    matrixforthis = matrix
    Queenpos = [[-1, -1] for _ in range(Queen)]
    iteration = 0

    def sameblock(a, b, n):
        for i in range(n):
            if Queenpos[i][0] == a and Queenpos[i][1] == b:
                return True
        return False

    def findPos(n):
        nonlocal iteration
        if n == num:
            iteration += 1
            if iteration % 1000 == 0:
                printSolution(Queenpos, matrix)
            return checkQueen()

        else:
            for i in range(Queen):
                for j in range(Queen):
                    if sameblock(i, j, n):
                        continue
                    else:
                        Queenpos[n][0] = i
                        Queenpos[n][1] = j
                        if findPos(n+1):
                            return True
            return False

    def checkQueen():
        for i in range(Queen):
            for j in range(i + 1, Queen):
                if Queenpos[i][0] == Queenpos[j][0]:
                    return False
                if Queenpos[i][1] == Queenpos[j][1]:
                    return False
                if abs(Queenpos[i][0] - Queenpos[j][0]) == 1 and abs(Queenpos[i][1] - Queenpos[j][1]) == 1:
                    return False
                if matrixforthis[Queenpos[i][0]][Queenpos[i][1]] == matrixforthis[Queenpos[j][0]][Queenpos[j][1]]:
                    return False
        return True

    if findPos(0):
        return Queenpos, iteration
    else:
        return None, iteration

```

Gambar 2. Kode Algoritma

BAB IV: TESTING

Bagian ini merupakan bagian untuk melakukan tes pada program, input untuk tes akan disimpan pada folder .../Tucil1_13524083/src/test dalam bentuk .txt. Hasil dari setiap test case akan disimpan pada .../Tucil1_13524083/src/test/output Anda juga bisa mencoba sendiri dengan tata cara dan petunjuk sudah terlampir pada file README.md.

4.1 Test case 1

- Input

```
AAB
ABB
CCC
```

- Output

```
No solution found
```

4.2 Test case 2

- Input

```
AABB
ACCD
BCCD
BDDD
```

- Output

```
No solution found
43680

Time taken: 0.0667 seconds
```

4.3 Test case 3

- Input


```
AAABB  
ACCDB  
ECCDB  
EEDDD  
EEEDD
```

- Output

```
QAABB  
ACQDB  
ECCDQ  
EQDDD  
EEEQD  
  
Queen positions:  
Queen 1: (0, 0)  
Queen 2: (1, 2)  
Queen 3: (2, 4)  
Queen 4: (3, 1)  
Queen 5: (4, 3)  
69593  
  
Time taken: 0.1087 seconds
```

4.4 Test case 4

- Input

```
AAABBC  
ADDBBC  
EDDEFC  
EEFFFC  
EEFFFC  
EEFFCC
```

- Output

```

QAABBC
ADDBQC
EDQEFC
EEFFQ
EQFFC
EEFQCC

Queen positions:
Queen 1: (0, 0)
Queen 2: (1, 4)
Queen 3: (2, 2)
Queen 4: (3, 5)
Queen 5: (4, 1)
Queen 6: (5, 3)
10430568

Time taken: 16.8133 seconds

```

4.5 Test case 5

- Input

```

AAABBD
ABBBBD
EEEBDD
EEBBDC
EFFDDC
FFFFEC

```

- Output

```
QAABBD  
ABQBBD  
EEEEQD  
EQBBDC  
EFFDDQ  
FFFQFC
```

```
Queen positions:
```

```
Queen 1: (0, 0)
```

```
Queen 2: (1, 2)
```

```
Queen 3: (2, 4)
```

```
Queen 4: (3, 1)
```

```
Queen 5: (4, 5)
```

```
Queen 6: (5, 3)
```

```
8266148
```

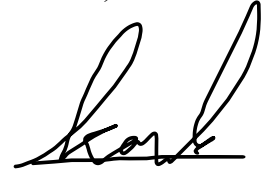
```
Time taken: 9.5130 seconds
```

LAMPIRAN

Github Repository

Program dapat diakses pada https://github.com/testbored/Tucil1_13524083

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Mahmudia Kimdaro Amin

Miscellaneous (Tabel Poin)

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki <i>Graphical User Interface</i> (GUI)		✓
6	Program dapat menyimpan solusi dalam bentuk file gambar		✓